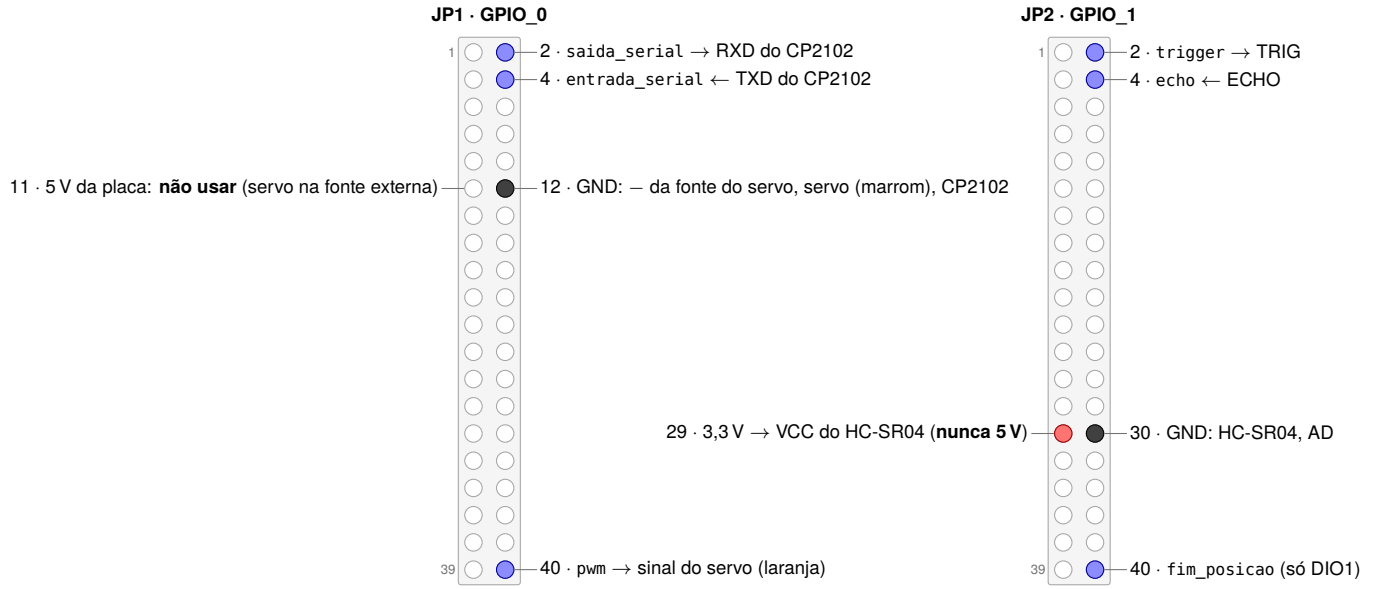
Cola de depuração — Sonar exp6 (PCS3645)

T1BA5 · top `sonar2` · 08/10/2026

Guia de montagem (DE0-CV)

Placa desligada até o passo 6. É a montagem da exp5 mais o TXD do CP2102. Conferir o pino 1 de cada conector pela marcação da placa.



● alimentação · ● GND (todos ligados entre si) · ● sinal · ○ não ligar. Ímpares à esquerda, pares à direita; sinais só na coluna par (2, 4, 40). A placa só alimenta o sensor (3,3 V, pino 29, vizinho do GND 30: um fio deslocado dá curto). O servo usa a fonte de 5 V externa (+ no VCC do servo, vermelho; − no GND comum, junto com o marrom do servo); o 5 V da placa (pino 11) fica livre.

Analog Discovery: cada fio do AD entra no mesmo ponto do sinal, em paralelo (coluna da protoboard ou pino livre), sem tirar o fio que já está lá.

Fio do AD	Cor*	Prender em	Sinal (pino FPGA)	Ver com
1+	laranja	sinal do servo (JP1 · 40)	`pwm` (T15)	Scope CH1
2+	azul	TRIG do HC-SR04 (JP2 · 2)	`trigger` (A12)	Scope CH2
DIO0	rosa	ECHO do HC-SR04 (JP2 · 4)	`echo` (B12)	Logic
DIO1	rosa	direto no pino JP2 · 40 (não vai em mais nada)	`fim_posicao` (K16)	Logic
DIO6	rosa	TXD do CP2102 (JP1 · 4)	`entrada_serial` (C16)	Logic / Protocol UART
DIO7	rosa	RXD do CP2102 (JP1 · 2)	`saida_serial` (B16)	Logic / Protocol UART
1−, 2−	laranja/branco, azul/branco	GND comum	referência dos canais	—
GND	preto	GND comum (JP2 · 30)	referência do AD	—

* Cores do cabo do Analog Discovery 2; conferir pelo rótulo do conector do cabo. Configurações de cada instrumento na seção Analog Discovery.

Montagem e energização

1. **GND primeiro:** GND da placa (pinos 12 e 30) no GND do sensor, do CP2102, do AD e no − da fonte externa do servo.
2. **Alimentação:** HC-SR04 em 3,3 V (JP2 · 29); em 5 V o echo chega a 5 V na FPGA. Servo na fonte externa de 5 V (+ no vermelho, − no marrom); nunca pelo 5 V da placa nem por pino de E/S.
3. **Sinais:** conforme o diagrama. TXD e RXD cruzados; fios do AD (tabela) em paralelo, sem interromper o fio.
4. **Mecânica:** sensor sobre o eixo do servo, girando livre de 20° a 160° sem puxar cabos; 90° apontando para a frente.
5. **Multímetro, placa desligada:** sem curto entre 3,3 V e GND, nem entre + e − da fonte.
6. **Energizar** a placa com SW0 = 0, SW1 = 0 e SW9..7 = 000; depois a fonte (5 V). Medir 3,3 V no VCC do sensor e 5 V no VCC do servo.
7. **Programar:** `make jtag` (USB-Blaster na lista) e `make program`.
8. **CP2102 no USB:** `ls /dev/ttyUSB*`; Processing ou `picocom`, um de cada vez.

Primeiro teste (só avançar se a linha anterior passou)

	Como	Esperado
EU1	CP2102 sem FPGA, TXD ligado ao RXD; `picocom`	cada tecla volta na tela
—	programado, SW1 = 0	HEX5 = 0; LEDR4 e LEDR5 acesos (linhas em repouso)
EU2	SW1 = 0, página 011; teclar `a`, `v`, `x`	HEX1 HEX0 = 61, 76, 78; paridade ok = 1; LEDR9 apagado
—	SW1 = 1	LEDR0 aceso; HEX5 HEX4 = `2 0`; servo em 020
EU3	Scope CH1 no `pwm`	20 ms; larguras da tabela do servo
EU4	objeto a 10 e 30 cm (régua), página 000	distância ±2 cm; trigger de 10 µs
EI1	Processing	`aaa,ddd#`; ângulos 020 → 160 → 020
EI2	teclar `a`, depois `v`	LEDR9 acende e o servo para no fim do ciclo; `v` retoma da posição mantida

Números-chave

O ciclo dura cerca de 2 s: espera, medida, envio de `aaa,ddd#` e, no modo 0, avanço do servo. Clock de 50 MHz (20 ns).

Grandeza	Valor	Origem
Espera entre medidas	2 s = 100 000 000 clocks	`ESPERA` em `sonar2.v`
Timeout da medida	10 s = 500 000 000 clocks	`TIMEOUT` em `sonar2.v`
Pulso de trigger	10 µs = 500 clocks	`gerador_pulso`
Echo → distância	58,8 µs por cm = 2941 clocks	`contador_cm` (R = 2941)
Echo máximo real (4 m)	cerca de 23,5 ms	HC-SR04
Sem eco (sensor desiste)	cerca de 47,5 ms → lê 807 cm	medido na bancada **(06/10)**
Serial	115200 bauds, 7 bits, paridade par, 1 stop (7E1)	TX e RX
Um caractere	10 bits = 86,8 µs	start + 7 + paridade + stop
Mensagem `aaa,ddd#`	8 caracteres, cerca de 700 µs	ângulo, vírgula, distância, #
PWM do servo	período 20 ms (50 Hz), pulso 0,7 a 2,2 ms	`controle_servo_8`
Sincronizadores (`ligar`, `echo`, `entrada_serial`)	2 FFs cada, atraso de 40 ns	`sincronizador.v` no `sonar2.v`
Simulação (testbenches)	espera 200 µs, timeout 20 ms	`sonar2_tb_config.vh`

Comandos: `a` = modo atenção (servo para), `v` = volta à varredura. Só minúsculos e com paridade correta; o resto é ignorado. Desligado (`ligar` = 0), os comandos são ignorados.

Chaves e LEDs

Chave	Sinal	Uso
SW0	`reset`	1 = reset assíncrono (estado 0, modo 0, posição 000)
SW1	`ligar`	1 = sonar funcionando; 0 = UC em `inicial`
SW9..7	`sel_depuracao`	página de HEX5..HEX0

LED	Sinal	O que indica
LEDR0	`db_ligar`	a chave SW1 direto (antes do sincronizador)
LEDR1	`db_echo`	echo do sensor
LEDR2	`db_trigger`	trigger (10 µs, quase invisível)
LEDR3	`db_pwm`	PWM do servo (brilho fraco)
LEDR4	`db_saida_serial`	linha TX; aceso em repouso
LEDR5	`db_entrada_serial`	linha RX; aceso em repouso se o TXD está ligado
LEDR8..6	`db_posicao`	posição do servo em binário
LEDR9	`db_modo`	aceso fora da localização (atenção ou modo novo); o número do modo está no HEX4 da pág. 011

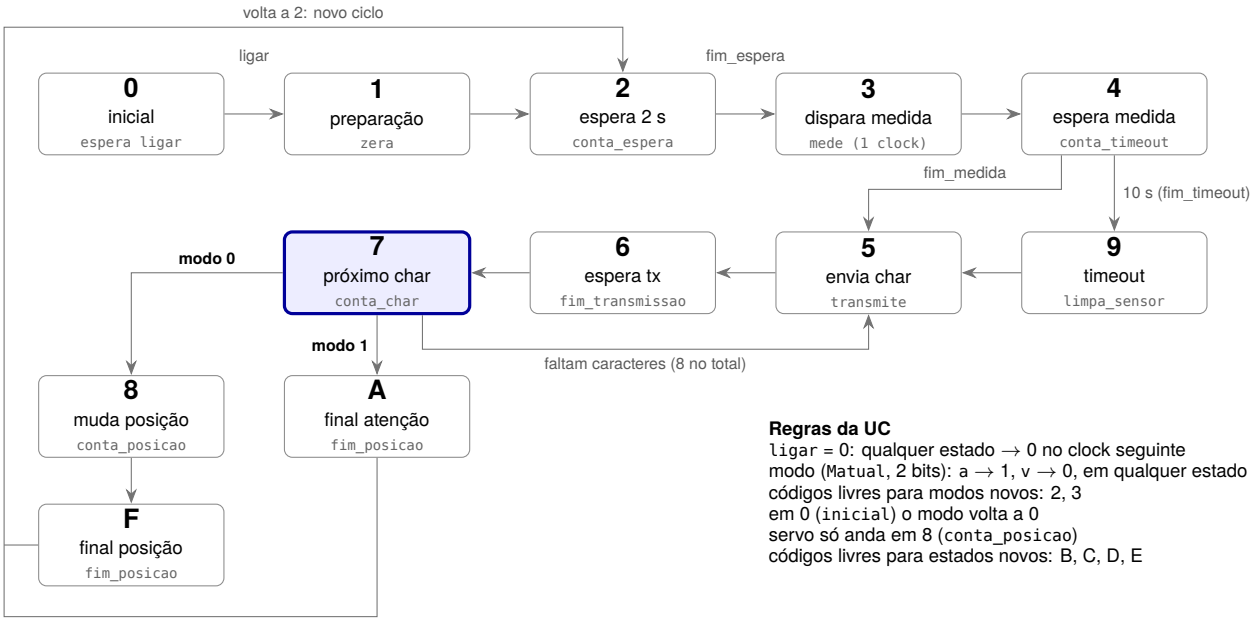
Páginas dos displays (SW9..7)

O multiplexador `U3_PAGINA` (`sonar2.v`) escolhe 1 de 8 páginas de 6 dígitos. HEX5 é o display mais à esquerda.

SW9..7	Página	HEX5	HEX4	HEX3	HEX2	HEX1 HEX0
000	sonar (= exp5)	estado UC	estado HC-SR04	posição 0–7	distância (centena)	distância (dez., unid.)
001	serial	estado TX	char TX (ASCII)	char TX (ASCII)	estado RX	char RX (ASCII)
010	mensagem	ângulo	ângulo	ângulo	distância	distância
011	comandos	estado UC	modo 0–3 (0 loc., 1 atenção, 2–3 livres)	índice do char 0–7	paridade ok	char RX (ASCII)
1xx	livres (prova)	0	0	0	0	0 0

Simulação (caso 20): 000 = `201050`, 001 = `030078`, 010 = `040050`, 011 = `200178`, 1xx = `000000`. ASCII: `a` = 61, `v` = 76, `A` = 41, `x` = 78, `0`–`9` = 30–39, `,` = 2C, `#` = 23. Em funcionamento normal, a página 000 mostra `2 0` em HEX5 HEX4 quase o tempo todo. Página nova: edite `s_pagina4` a `s_pagina7` em `sonar2.v` (6 dígitos hex, 24 bits).

Códigos de estado



Sinais de cada estado (`sonar2_uc`, Moore: cada sinal de controle fica em 1 só no seu estado; nos demais, todos em 0).

Cód.	Estado	Sinal da UC em 1 → efeito no FD	Sai para (entrada da UC)	Nos pinos e LEDs
0	`inicial`	nenhum; modo forçado a 0	1 se `ligar` = 1	servo parado na posição atual; `saida_serial` = 1 (repouso)
1	`preparacao`	`zera`: posição (U4) → 000, char (U6) → 0, espera (U8), TX e sensor reiniciados	2 (1 clock)	`pwm` vai a 0,7 ms (020°); LEDR8..6 = 000
2	`espera`	`conta_espera`: U8 conta 2 s	3 com `fim_espera` (U8)	tudo em repouso; servo se acomoda
3	`dispara_medida`	`mede`: U1 inicia a medida; zera o timeout (U9)	4 (1 clock)	`trigger`: pulso de 10 µs
4	`espera_medida`	`conta_timeout`: U9 conta 10 s	5 com `fim_medida` (pronto do U1); 9 com `fim_timeout` (U9)	`echo` em 1 por 58,8 µs/cm; HEX4 (sensor) 3 → 4 → 5
9	`timeout_medida`	`limpa_sensor`: reinicia U1 (sai de espera echo)	5 (1 clock)	mensagem sai com distância 000
5	`envia_char`	`transmite`: partida do TX (U2) com o char do mux (U7)	6 (1 clock)	`saida_serial` desce (start bit)
6	`espera_tx`	nenhum	7 com `fim_transmissao` (pronto do U2)	`saida_serial`: quadro de 86,8 µs
7	`proximo_char`	`conta_char`: U6 avança o índice (0–7)	5 se falta char; com `fim_char`: 8 (modo 0) ou A (modo 1)	bifurcação; HEX3 da pág. 011 = índice
8	`muda_posicao`	`conta_posicao`: U4 avança (111 → 000)	F (1 clock)	`pwm` muda de largura: servo anda 20°; LEDR8..6
F	`final_posicao`	`fim_posicao`	2 (1 clock)	pino `fim_posicao`: pulso de 20 ns
A	`final_atencao`	`fim_posicao`	2 (1 clock)	pulso de 20 ns; servo não anda

Sempre ativos, fora da UC: `pwm` (U3, 20 ms, largura pela posição), recepção (U10) e FSM de modo. `ligar` = 0 leva a 0 de qualquer estado; `reset` leva a 0 e ao modo 0. Saídas do topo: `trigger`, `pwm`, `saida_serial`, `fim_posicao`, `db_modo` (LEDR9), HEX5..0 e LEDR8..0.

Máquina	Onde aparece	Códigos
HC-SR04 (`interface_hcsr04_uc`)	HEX4, pág. 000	0 inicial · 1 preparação · 2 envia trigger · 3 espera echo · 4 medida · 5 armazenamento · F final
TX (`tx_serial_uc`)	HEX5, pág. 001	0 inicial · 1 preparação · 3 espera · 7 transmissão · F final
RX (`rx_serial_uc`)	HEX2, pág. 001	0 inicial · 2 preparação · 3 espera · 7 recepção · 9 registra · F final

Servo: posição, ângulo e PWM

Posição	Ângulo	Largura	Clocks	ASCII da ROM
000	020	0,700 ms	35 000	30 32 30
001	040	0,914 ms	45 700	30 34 30
010	060	1,129 ms	56 450	30 36 30
011	080	1,343 ms	67 150	30 38 30
100	100	1,557 ms	77 850	31 30 30
101	120	1,771 ms	88 550	31 32 30
110	140	1,986 ms	99 300	31 34 30
111	160	2,200 ms	110 000	31 36 30

A posição (contador U4) escolhe a largura do pulso e o ângulo da ROM. LEDR8..6 mostram a posição em binário.
No modo 0 a varredura vai 000 → 111 e volta a 000 (dente de serra).
Período de 20 ms (1 000 000 clocks). Larguras em `controle_servo_8.v`, ângulos em `rom_angulos_8x24.v`.

Analog Discovery / WaveForms

Canais conforme o planejamento. Todos os sinais em 3,3 V do lado da FPGA; GND do AD no GND da placa.

Sinal	Canal	Ferramenta e configuração	O que esperar
`entrada_serial`	DIO6	Logic + UART ou Protocol: 115200, 7 bits, Even, Standard, LSB first; trigger ↓; 20 µs/div	quadro de 86,8 µs; a = 61, v = 76
`saida_serial`	DIO7	mesma configuração; 100 µs/div	`aaa,ddd#` em cerca de 700 µs
`pwm`	CH1+	Scope: subida, 1,5 V; 500 µs/div (largura) ou 5 ms/div (período)	20 ms; larguras da tabela do servo
`trigger`	CH2+	Scope: captura única, subida; 5 µs/div	pulso de 10 µs
`echo`	DIO0	Logic: subida; 2 ms/div (10 ms/div para o caso sem eco)	58,8 µs/cm; cerca de 47 ms quando lê 807
`fim_posicao`	DIO1	Logic: 100 MHz; modo Record para vários ciclos	pulso de 20 ns a cada 2 s

Trigger e echo juntos no Scope (usado na bancada em 08/10): mostra se o sensor responde ao disparo e a largura do echo. Mudar o 1+ do `pwm` para o ECHO (JP2 · 4); o 2+ continua no TRIG.

Campo	Valor	Campo	Valor
Mode / Trigger	Repeated / Auto	Time Base	5,8 ms/div
Source	Channel 1 (`echo`)	Time Position	4,971 ms (disparo perto da esquerda)
Type / Condition	Edge / Rising	Channel 1 e 2	Offset 0 V, Range 1 V/div
Level / Hyst. / HoldOff	2 V / Auto / 0 s	Amostragem	8192 amostras a 141 kHz

O que se vê: o trigger (CH2, azul) é um traço fino logo antes da subida do echo (10 µs some a 5,8 ms/div; para medir, 5 µs/div); o echo (CH1, laranja) é um pulso de cerca de 3,3 V com 58,8 µs por cm (3,4 ms ≈ 57 cm). A janela vai até cerca de 34 ms depois do disparo: cobre 4 m (23,5 ms), mas não o caso sem eco (47 ms; usar 10 ms/div). Echo em 0 depois do traço do trigger = sensor não respondeu (alimentação, GND ou fio do ECHO).

- **Texto decodificado:** no UART do Logic, Format = ASCII. **Congelar:** Mode = Single. **Salvar figura:** File → Export.
- **Comando sem o CP2102:** Protocol → UART, TX no DIO6, Send a (antes, desligar o TXD do CP2102 do JP1 pino 4).
- **Servo no tempo:** Data Logger, CH1 no `pwm`, função `C1DC / 3.3 * 20` (largura em ms): escada da varredura e patamar no modo atenção.
- **Distância:** Scope com o `echo` num canal analógico, Measurements → PosWidth; Custom `Channel.measure("PosWidth", T1, T2) / 58e-6` dá cm.
- Scope e Data Logger usam os mesmos canais analógicos: um de cada vez.

Sintoma → causa → ação

Ordem: alimentação e fios, chaves e páginas, Analog Discovery, comparação com a simulação. **(06/10)** = aconteceu na bancada.

Sintoma	Causa provável	Ação
Nada acontece, HEX5 = 0	SW1 = 0 ou SW0 = 1	Ajustar as chaves; LEDR0 aceso = ligar ok
Mensagens chegam, mas a não muda nada **(06/10)**	Fio TXD → JP1 pino 4 solto ou no pino errado	LEDR5 aceso em repouso; página 011 deve mostrar 61; trocar o jumper
a chega (61), paridade ok = 0	Terminal fora de 7E1	115200, 7 bits, paridade par
LEDR9 acende, mas o servo anda mais uma posição	a chegou depois da bifurcação	Previsto: para no fim do ciclo seguinte
Comando ignorado com o sonar desligado	Previsto: em `inicial` o modo fica em 0	Ligar antes de enviar
HEX5 HEX4 = `3 0` parado após desligar/religar SW1 **(06/10)**	SW1 assíncrona + UC one-hot sem safe state → estado ilegal	Corrigido com 2 FFs em `ligar`, `echo` e `entrada_serial` (`sincronizador.v`); se voltar, reset (SW0)
Leitura de 807 cm em algumas posições **(06/10)**	Sensor sem eco: segura o echo cerca de 47 ms	Objeto plano e rígido a menos de 4 m; Processing mostra “Não Detectado”
HEX5 = 4, HEX4 = 3 por 10 s, depois `aaa,000#`	Sensor não respondeu (timeout)	Conferir 3,3 V (JP2 · 29), GND e fio do echo
HEX5 = 6 preso	Falha na transmissão	Analisar `saida_serial` no DIO7
Servo parado ou tremendo	Queda de tensão do servo	Verificar 5 V da fonte externa e o − dela no GND comum
`Device or resource busy`	Outro programa usa a porta	Fechar Processing ou picocom
Processing não envia a tecla	Foco no editor	Clicar na janela do radar
`A` maiúsculo não funciona	Caps Lock: `A` = 41 é ignorado	Desligar Caps Lock
`No JTAG hardware available`	Cabo USB da placa desligado	Ligar o cabo; `make jtag`

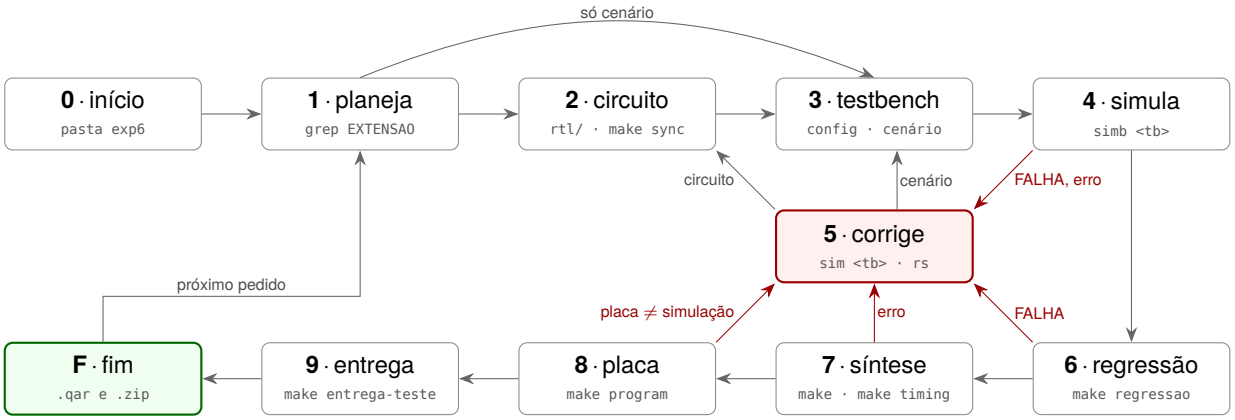
Comandos

Comando (em `labdig2/exp6`)	O que faz
`make check TB=sonar2_tb`	Icarus (cerca de 1 min)
`make simb TB=sonar2_tb`	Questa batch; “todos os casos OK”
`make sim TB=sonar2_tb`	Questa com ondas
`sim cenario_m_tb`	igual a `make sim TB=...`; aceita o módulo ou `tb/cenario_m_tb.v` (Tab), não `cenario_m_tb.v`; `simb`/`check` direto têm limite de 60 s (`sonar2_tb` no Icarus: use `make check`)
`make regressao`	todos os `*_tb.v` no Questa, com resumo
`make`	compila (`.sof` + timing)
`make timing`	relatório de timing
`make jtag` / `make program`	lista o USB-Blaster / grava
`make gui`	Quartus (Pin Planner, RTL Viewer)
`make entrega`	`.qar` e zip do ModelSim em `entrega/`
`make entrega-teste`	roda o zip extraído (antes de entregar)

Terminal serial (fechar o Processing antes):
`ls /dev/ttyUSB*`
`picocom -b 115200 -d 7 -y e -p 1 /dev/ttyUSB0`
`-y` = paridade, `-p` = stop bits. Ctrl+A Ctrl+X sai; Ctrl+A Ctrl+C liga o eco local. Teste de paridade errada: 8N1 (`-d 8 -y n`) e teclar `v`: o modo não muda.
Processing: porta `/dev/ttyUSB0`, já em 7E1. Clicar na janela do radar e teclar; o console mostra `Enviando tecla 'a' ...`. Acima de 100 cm aparece “Não Detectado”; só abaixo de 50 cm vira ponto no radar.

Workflow como máquina de estados: notebook (Linux)

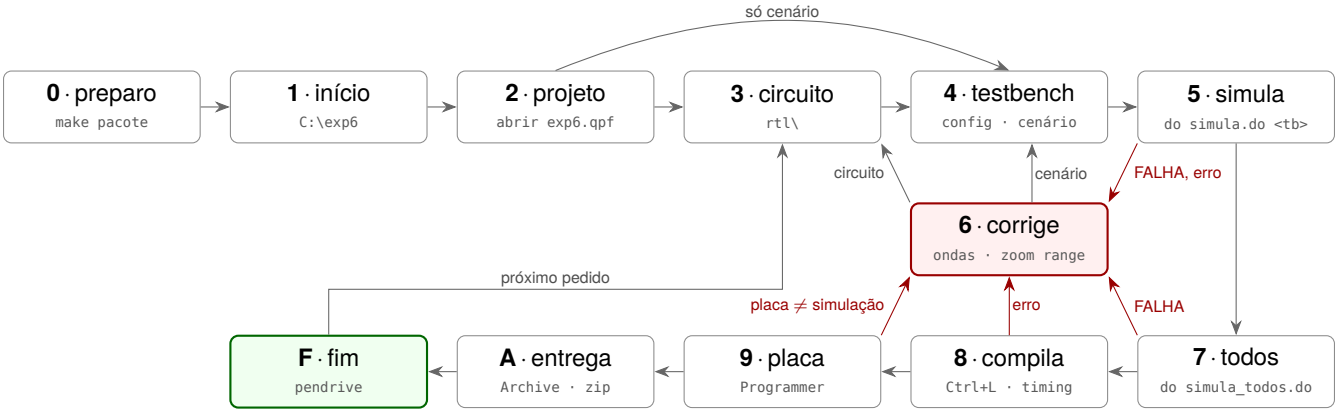
Máquina de Moore: em cada estado você executa a saída (os comandos da tabela) e o resultado escolhe a transição. Seta preta: deu certo. Seta vermelha: deu errado, vai para **5 corrige**, que devolve ao circuito ou ao testbench. Terminal sempre na pasta `exp6`; `<tb>` = módulo ou caminho do arquivo.



Estado	O que fazer	Vai para
0 início	terminal na pasta: `cd ~/projetos/labdig2/exp6`	1
1 planeja	ler o pedido; `grep -n EXTENSAO rtl/*.v *.vh wave.do`; guia de modificações da cola	2 se muda o circuito; 3 se é só cenário
2 circuito	editar `rtl/` nos pontos `EXTENSAO`; arquivo novo em `rtl/`: `make sync`	3
3 testbench	circuito mudou: `sonar2_tb_config.vh`; cenário: `cp tb/cenario_m_tb.v tb/cenario_x_tb.v` e trocar o nome do módulo	4
4 simula	`simb <tb>` (ou `make simb TB=cenario_x_tb`); 6 s por cenário	6 se “todos os casos OK”; 5 se FALHA ou erro de compilação
5 corrige	`sim <tb>`: ondas com o `wave.do`; colar o `wave zoom range` do relatório; `rastro_uc = 1;` no cenário; editar e digitar `rs` no Questa	2 se o erro é do circuito; 3 se é do cenário
6 regressão	`make regressao`: todos os `*_tb.v`, resumo no fim	7 se 0 com falha; 5 se não
7 síntese	`make` e `make timing`	8 se TIMING PASS; 5 se erro
8 placa	`make jtag`, `make program`; SW0 = reset, SW1 = ligar, SW9..7 = página; tabela sintoma → causa → ação	9 se funciona; 5 se a placa difere da simulação
9 entrega	`make entrega-teste` (zip extraído roda?); depois `make entrega` (`entrega/`)	F se OK; 5 se o zip falha
F fim	enviar o `.qar` e o `.zip`; para o PC da faculdade: `make pacote`	1 no próximo pedido

Workflow como máquina de estados: PC da faculdade

Sem `make`: Quartus e ModelSim/Questa pela interface. A pasta vem do notebook (`make pacote`): o Quartus abre o projeto dela e o ModelSim simula a partir dela, então os dois usam a mesma cópia do RTL. Mesma convenção de setas.



Estado	O que fazer	Vai para
0 preparo	no notebook: `make pacote` → `entrega/exp6_pasta_t1ba5.zip` no pendrive (projeto + testbenches + cola). Reserva: `make entrega` (`.qar` e zip do ModelSim)	1
1 início	extrair numa pasta sem espaço nem acento, ex.: `C:\exp6`; Quartus: Help → About mostra a versão	2
2 projeto	Quartus: File → Open Project → `quartus\de0_cv\exp6.qpf`. Se não abrir (versão mais antiga): Project → Restore Archived Project com o `.qar`, ou New Project Wizard (Cyclone V 5CEBA4F23C7, top `sonar2`) + arquivos de `rtl` + Assignments → Import Assignments (`exp6.qsf`)	3 se muda o circuito; 4 se é só cenário
3 circuito	editar `rtl\` nos pontos `EXTENSA0` (editor do Quartus ou Notepad++); arquivo novo: Project → Add/Remove Files in Project	4
4 testbench	circuito mudou: `sonar2_tb_config.vh`; cenário: copiar `tb\cenario_m_tb.v` e trocar o nome do módulo	5
5 simula	ModelSim: File → Change Directory → `C:\exp6`; no Transcript: `do simula.do cenario_x_tb`	7 se “todos os casos OK”; 6 se FALHA ou erro
6 corrige	ondas já organizadas pelo `wave.do`; colar o `wave zoom range` do relatório; editar e repetir `do simula.do cenario_x_tb`	3 se o erro é do circuito; 4 se é do cenário
7 todos	`do simula_todos.do` (resumo no fim do Transcript)	8 se 0 com falha; 6 se não
8 compila	Quartus: Processing → Start Compilation (Ctrl+L); Compilation Report → Timing Analyzer: setup e hold sem slack negativo	9 se OK; 6 se erro
9 placa	Tools → Programmer → Hardware Setup: USB-Blaster; Mode JTAG; `output_files\exp6.sof`; Start	A se funciona; 6 se a placa difere da simulação
A entrega	Project → Archive Project (`exp6_t1ba5.qar`); zip com `rtl`, `tb`, `*.vh` e `*.do`; testar: extrair numa pasta nova e `do simula_todos.do`	F se OK; 6 se não
F fim	pasta de volta no pendrive (o notebook não vê o que mudou aqui)	3 no próximo pedido

Testbench: como modificar (prova)

Arquivo	O que tem	Mexer quando
`tb/cenario_x_tb.v`	o roteiro: casos, comandos, ciclos e conferências, em sequência	sempre (cenário novo)
`sonar2_tb_config.vh`	tudo o que depende do circuito: DUT (portas e parâmetros), tempos da simulação, códigos dos modos, pulsos `recebeu_*`, mensagem esperada, regra do `ANDA`, onde o modo aparece	o circuito mudou (procurar `EXTENSAO`)
`sonar2_tb_base.vh`	clock, serial (lê e escreve), sensor, conferências, log e relatório da troca de modo	nunca
`wave.do`	ondas de todos os cenários; nomes de estados e modos lidos da UC; pulsos `s_recebeu_*` entram sozinhos	sinal novo nas ondas (fim do arquivo)

Estado ou modo novo na UC não muda nada no testbench nem nas ondas: os nomes saem das linhas `parameter nome = 4'hX;` (estado) e `parameter NOME = 2'dN;` (modo) da UC. No cenário, um estado pelo nome: `espera_estado(dut.U1_UC.muda_posicao)`.

Esqueleto (`cp tb/cenario_m_tb.v tb/cenario_x_tb.v` e trocar o nome do módulo)

```
`timescale 1ns/1ns
module cenario_x_tb;
`include "sonar2_tb_base.vh"
  initial begin
    inicia;                 // reset, liga
    novo_caso(1, "localizacao");
    ciclo(0, 15, ANDA);
    novo_caso(2, "'a' na espera");
    envia_comando("a", ATENCAO);
    ciclo(1, 25, PARADO);
    finaliza;               // relatorio
  end
endmodule
```

`make check TB=cenario_x_tb` (Icarus, 10–40 s)
`make sim TB=cenario_x_tb` (Questa, ondas)
`make regressao` (todos; acha o cenário novo sozinho)

Regra: cada linha começa quando a anterior termina, e as tasks `espera_*` param até o evento. Para um comando chegar num momento do ciclo, escreva a linha dele logo depois da task que espera aquele momento.

Momento	Linhas do cenário
na espera	`envia_comando("a", ATENCAO); ciclo(p, cm, PARADO);`
na medida	`espera_trigger; envia_comando(...); ciclo(...);`
na transmissão	`responde_echo(cm); espera_transmissao; envia_comando(...); fim_ciclo(p, cm, ...);`
num estado	`responde_echo(cm); espera_estado(dut.U1_UC.x); envia_comando(...); fim_ciclo(...);`
com o echo em 1	`fork responde_echo(cm); begin wait (echo_in); #(...); envia_comando(...); end join` (caso 12 dos exemplos)

Antes da bifurcação (espera, medida, transmissão) o comando já vale no próprio ciclo. Vários comandos seguidos podem passar da espera de 200 µs: o `ciclo` responde ao trigger que já veio. Padrões prontos: `tb/cenario_exemplos_tb.v`.

Task	O que faz
`inicia` · `finaliza`	reset e liga · desliga, relatório das trocas de modo e resultado
`novo_caso(n, "texto")`	número e texto do caso no log e nas ondas (`caso`, `caso_texto`)
`ciclo(p, cm, depois)`	trigger, echo de `cm`, `fim_posicao`; confere `aaa,ddd#` e o servo em `depois`
`depois`	`ANDA` (regra da config), `PARADO` ou a posição exata (0–7)
`responde_echo(cm)` + `fim_ciclo(p, cm, depois)`	o mesmo ciclo em duas metades, para pôr algo no meio
`medida(p, cm)`	echo e mensagem conferida, sem esperar o fim do ciclo (várias medidas por posição)
`ciclo_sem_echo(p, depois)`	sensor não responde: `aaa,000#` e duração $\geq$ `T_TIMEOUT`
`envia_comando(c, modo)`	quadro 7E1 (87 µs) e confere o modo na UC, no LEDR9 e no HEX4 da pág. 011
`envia_comando_paridade_errada(c, modo)`	quadro com a paridade invertida; confere que o modo não mudou
`envia_char(c, PARIDADE_OK)`	só o quadro, sem conferir
`espera_trigger` · `espera_transmissao` · `espera_mensagem` · `espera_fim_ciclo` · `espera_estado(e)`	param até o evento da medida atual; voltam na hora se ele já aconteceu
`liga` · `desliga` · `aplica_reset`	`ligar` = 1 · `ligar` = 0 e espera 100 µs · pulso de 2 µs no reset
`confere_mensagem(p, cm)` · `confere_posicao(p)` · `confere_modo(m)` · `confere_estado(e)`	conferências avulsas
`confere_pagina(sel, 24'hXXXXXX)`	os 6 dígitos de uma página; depois volta à página anterior
`confere_sinal("nome", sinal, valor)`	qualquer sinal: LED, GPIO novo, interno (`dut.s_...`)
`confere_tempo("nome", t, min, max)`	intervalo em ns; `t_trigger`, `t_trigger_ant`, `t_fim_ciclo`, `T_ESPERA`, `T_TIMEOUT`, `T_QUADRO`
`rastro_uc = 1;`	imprime cada transição da UC, com o nome do estado

O circuito mudou: o que mexer no testbench

Mudança no circuito	No testbench
Estado novo na UC	nada (nome lido da UC; ondas também)
Modo novo (código 2 ou 3)	config: `localparam DUPLO = 2;`
Comando novo (`s_recebeu_x`)	config: `always @(posedge dut.s_recebeu_x) registra_comando("recebeu_x");` Ondas: nada
Caractere de comando trocado	cenário: `envia_comando("N", ...)`
Parâmetro novo no `sonar2`	config: `localparam X_SIM` reduzido (2 s → 200 µs: $\div 10^4$) e `.X(X_SIM)` no DUT
Porta nova (LED, GPIO)	config: `wire x_out;` e `.x(x_out)` no DUT; cenário: `confere_sinal("x", x_out, 1);`
Movimento diferente	config: `posicao_seguinte`; ou a posição exata no `ciclo(p, cm, 6)`
Várias medidas por posição	cenário: `medida(p, cm1); medida(p, cm2); espera_fim_ciclo; confere_posicao(p+1);`
Intervalo diferente	cenário: `t0 = t_fim_ciclo; espera_trigger; confere_tempo("intervalo", t_trigger - t0, min, max);`
Formato da mensagem	config: `TAM_MSG` e `mensagem_esperada`
Página nova nos displays	cenário: `confere_pagina(3'b100, 24'h...);`
Modo mostrado em outro lugar	config, seção 7 (LED e dígito do display)

- **Item "localize a troca de modo":** o `finaliza` imprime, para cada troca, `rx_pronto` (dado e paridade), `recebeu_*`, a troca de `Matual` e as transições da UC do fim do último ciclo no modo antigo até o 1º `dispara_medida` no novo, mais `wave zoom range …` para colar no Questa. O `wave.do` abre com o grupo Troca_de_modo no topo.
- **Icarus x Questa:** no `make check`, `gerador_pulso_tb` e `interface_hcsr04_tb` falham por um artefato do Icarus: o `gerador_pulso` faz `pulso = 0` e depois `1` no mesmo `always @(*)`, e o trigger ganha bordas de largura zero a cada clock. No Questa (`make simb`, `make regressao`) os dois passam. Nos cenários, esperar o trigger com `espera_trigger`, nunca com `@(posedge trigger_out)`.
- **Cenário com erro de sintaxe** quebra o `make check`, `make simb` e `make sim` de *todos* (eles compilam `tb/` inteiro): corrigir ou tirar de `tb/`. O `make regressao` e o `do simula.do <tb>` isolam o arquivo e rodam os outros.
- **Tempos na simulação:** espera 200 µs, timeout 20 ms, quadro 87 µs, echo 400 µs depois do trigger, `cm` ≥ 1. O `sonar2_tb` leva cerca de 1 min no Icarus.
- **Entrega:** `make entrega-teste` monta o zip do ModelSim, extrai numa pasta temporária e roda todos os testbenches, como quem corrige.

Guia rápido de modificações (circuito)

Regra: o FD só detecta e executa; a UC decide. `grep -n EXTENSAO rtl/*.v *.vh wave.do` lista todos os pontos de mudança. Depois de mudar: `make check TB=...`, `make regressao`, `make`, `make timing`; arquivo novo em `rtl/`: `make sync`.

Pedido do desafio	Onde	Como
Trocar os caracteres dos comandos	`sonar2.v`	`CHAR_ATENCAO = "N"`, `CHAR_VOLTAR = "A"`; no cenário, `envia_comando("N", ...)`
Comando novo que muda o modo (ex.: `d`)	topo, FD, UC, tb	topo: parâmetro `CHAR_DUPLO`, fio `s_recebeu_duplo`, ligar na UC e no FD (parâmetro e porta). FD: parâmetro, porta e um comparador igual aos outros. UC: porta, `parameter DUPLO = 2'd2;`, transições no `case (Matual)`, ramo na bifurcação. tb: `localparam DUPLO = 2;` e o `always` do pulso
Estado novo	`sonar2_uc.v`	código livre B–E; transição no `case`; saída no bloco de Moore; estado final de um modo entra no `fim_posicao`. tb e ondas: nada
Várias medidas por posição	UC	flag na UC que marca a 1ª medida; na bifurcação: 1ª medida → estado novo → `espera`; 2ª → `muda_posicao` (flag volta a 0)
Intervalo diferente num modo (ex.: 1 s)	FD + UC	cópia do U8 com parâmetro novo (o `$clog2` acerta os bits) e `conta_*` próprio vindo da UC; estado de espera só do modo
Varredura vai-e-volta	só FD	U4 módulo 14; `s_posicao = (s_passo < 8) ? s_passo[2:0] : 14 - s_passo;` tb: posição exata no `ciclo`
Ação única (ex.: `z` leva o servo a 000)	FD + UC	comparador `recebeu_z`; FF de pedido na UC; na bifurcação, estado que ativa `zera_posicao` (também em `preparacao`)
Sinal novo nos displays	`sonar2.v`	`s_pagina4..7`; sinal interno do FD vira saída `db_...`
LED ou GPIO novo	`sonar2.v` + `.qsf`	porta no topo, pino no Pin Planner (`make gui`); tb: porta no DUT da config
Entrada assíncrona nova	`sonar2.v`	uma instância do `sincronizador` antes de ligar na UC ou no FD
Tratar "sem eco" (807 cm)	FD	comparador na distância ($>$ 400 cm) que substitui o valor enviado

O modo já tem 2 bits em todo o caminho (UC → topo → HEX4 da pág. 011; LEDR9 = modo $\neq 0$): um modo novo não muda a largura de nada. Os contadores de tempo usam `N = $clog2(M)`: qualquer tempo cabe (com `N` fixo, um `M` maior que 2^N travava a placa e passava na simulação).

Ensaio (08/10): o modo duplo (`d`, 2 medidas por posição) feito só nos pontos `EXTENSAO`: 15 edições no RTL, 2 linhas na config e um cenário de 20 linhas; passou de primeira, com os nomes do estado e do modo novos no log e nas ondas.

Decisões de projeto (perguntas prováveis)

Pergunta	Resposta curta
Por que a troca de modo não perde comandos?	A FSM de modo (`Matual`) é atualizada pelo pulso `recebeu_*` em qualquer estado, até no meio da medida ou da transmissão.
Por que a bifurcação fica no fim do ciclo?	A única diferença entre os modos é o reposicionamento, que já era o último passo. O ciclo em andamento termina de medir e transmitir.
Onde o servo para no modo atenção?	Na UC: o modo 1 pula `muda_posicao`. Não há porta lógica sobre o `pwm` nem sobre sinais de controle.
Por que conferir a paridade?	Um quadro corrompido poderia virar `a` ou `v`; quadros com paridade errada são descartados.
Por que `fim_posicao` pulsa também em atenção?	Marca o fim de cada ciclo; o tb e o Analog Discovery usam o pulso para sincronizar.
O que acontece ao religar?	`inicial` força o modo 0 (como o reset) e `preparacao` zera a posição.
Por que as entradas assíncronas passam por 2 flip-flops?	O Quartus codifica as FSMs em one-hot sem safe state; uma entrada que muda perto da borda do clock pode ser vista diferente por FFs diferentes e deixar a FSM num estado ilegal (HEX5 parado em 3 na bancada, 06/10). O `sincronizador` entrega um valor único por clock e dá tempo para a metaestabilidade se resolver; vale para `ligar`, `echo` e `entrada_serial`.
O atraso de 40 ns atrapalha?	Não: o echo atrasa nas duas bordas (a largura não muda) e 40 ns são 0,5 % de um bit da serial.
Por que o modo tem 2 bits se há 2 modos?	Reserva para a prova: um modo novo só muda a UC. O LEDR9 acende fora da localização e o HEX4 da pág. 011 mostra o número.
Por que 807 cm?	É o pulso de cerca de 47 ms que o sensor gera sem eco; o circuito mede corretamente.
Recursos do projeto	259 ALMs (1%), 331 registradores, 64 pinos; pior setup 15,14 ns, pior hold 0,149 ns; Fmax 214 MHz.